HAND-HYGIENE COMPLIANCE TIMER WITH DROPOUT TOLERANCE AND WASH LOGGING

Project Work for Integrated Skill Enhancement-1

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

By

Student Name	Roll Number
Srujana	24EG104E33

Department of Electronics and Communication Engineering

ANURAG UNIVERSITY SCHOOL OF ENGINEERING

Venkatapur (V), Ghatkesar (M), Medchal-Malkajgiri Dist-500088

2026-2027

ANURAG UNIVERSITY SCHOOL OF ENGINEERING
Venkatapur (V), Ghatkesar (M), Medchal-Malkajgiri Dist-500088

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

CERTIFICATE

This is to certify that the project report entitled **"Hand-Hygiene Compliance Timer with Dropout Tolerance and Wash Logging"** being submitted by

Srujana 24EG104E33

in partial fulfilment for the award of the Degree of Bachelor of Technology in Electronics & Communication Engineering to the Anurag University, Hyderabad, is a record of bonafide work carried out by the above student in the Department of Electronics and Communication Engineering. The results embodied in this project report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

Dr. Harikrishna Kamatham
Professor, HOD
ECE & ECM

External Examiner

DECLARATION

I hereby declare that the project report entitled **"Hand-Hygiene Compliance Timer with Dropout Tolerance and Wash Logging"**, submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering at Anurag University School of Engineering, is a record of original work carried out by me.

The work has not been submitted to any other university or institute for the award of any degree or diploma. Where I used an AI assistant to help with code skeletons and explanations, I reviewed, linted, simulated and understood every module, and the results reported here come from tools that I ran myself.

Srujana 24EG104E33

Date: 24/10/2026

Place: Hyderabad

ACKNOWLEDGEMENTS

I am grateful to Dr. Harikrishna Kamatham, Head of the Department of Electronics and Communication Engineering, for providing the facilities and encouragement I needed, and to all the faculty members of the department for what they have taught me over the years.

I also thank the open-source community behind Icarus Verilog, Verilator, Yosys, OpenROAD, OpenLane, KLayout and the SkyWater SKY130 PDK. Without these free tools, taking a design all the way to a GDS layout would not have been possible for a student.

Finally, I thank my family and friends for their support and for putting up with me while I waited for simulations and place-and-route runs to finish.

ABSTRACT

Incomplete hand washing is one of the cheapest infection risks to fix and one of the hardest to notice. People cut washes short, nobody is watching, and manual audits catch only a small share of what happens at the sink. This project designs a small digital chip that sits at a sink or sanitiser station and times every wash. The chip reads one hand-presence sensor signal, cleans it with a two-flip-flop synchronizer and a debounce counter, and runs a six-state finite state machine that starts a wash, counts seconds, shows progress in thirds, tolerates short dropouts through a grace period, and decides whether the wash was long enough. It offers three selectable durations (20, 30 and 40 seconds), raises a done or too-short indication with a buzzer signal, and keeps two eight-bit saturating counters of good and bad washes for auditing.

The design is written in synthesizable Verilog and verified with a self-checking testbench in which all 16 checks pass. Yosys reports 457 generic cells, 65 flip-flops and no latches. Using the open-source OpenLane flow on the SkyWater 130 nm process, the design was taken to a final GDS layout of about 104 µm × 115 µm. Static timing analysis at nine corners (three PVT conditions with three parasitic extremes each) shows positive setup and hold slack everywhere at a 20 ns clock, with a worst setup slack of 6.346 ns and a worst hold slack of 0.105 ns. DRC, LVS and antenna checks are clean. The work shows the complete path from a real-world problem to a verified, signed-off layout.

Keywords: hand hygiene, finite state machine, RTL to GDS, SKY130, OpenLane, multi-corner static timing analysis.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION **1**
1.1 Introduction 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Literature Survey 2
1.5 Organisation of the Report 3
CHAPTER 2: METHODOLOGY **4**
2.1 System Overview 4
2.2 Software and Hardware Requirements 4
2.3 Design Specification 5
2.4 Block Diagram 6
2.5 Module Description 7
2.6 FSM and Flowchart 8
2.7 Design Decisions 9
2.8 Verification Methodology 9
2.9 RTL-to-GDS Implementation Flow 10
2.10 Use of AI Assistance 10
CHAPTER 3: RESULTS AND DISCUSSION **12**
3.1 Functional Simulation Results 12
3.2 Synthesis Results 13
3.3 Physical Design Results 13
3.4 Multi-Corner Static Timing Analysis 15
3.5 Sign-off Checks 16
3.6 Discussion 16
3.7 Cost Analysis 17
3.8 Usefulness to Society 17
3.9 Environmental Safety 17
CHAPTER 4: CONCLUSION AND FUTURE SCOPE **18**
4.1 Conclusion 18
4.2 Future Scope 18
REFERENCES **19**
APPENDIX A: wash_fsm.v (RTL listing) **21**
APPENDIX B: OpenLane configuration and SDC **23**
APPENDIX C: Commands to reproduce the results **22**

LIST OF FIGURES

Figure 2.1: Block diagram of the hand-hygiene compliance timer 6
Figure 2.2: State diagram of the wash FSM 8
Figure 2.3: RTL-to-GDS design flow 10
Figure 3.1: Simulated waveform of a compliant wash 13
Figure 3.2: Final GDS layout of the design (full die) 14
Figure 3.3: Zoomed view of the layout showing cell rows and routing 15

LIST OF TABLES

Table 2.1: Software tools used in the project 4
Table 2.2: Off-chip components the chip is designed to work with 5
Table 2.3: Design parameters 5
Table 2.4: Port description 6
Table 2.5: FSM state table 8
Table 2.6: OpenLane configuration and timing constraints 10
Table 3.1: Testbench checks and results 12
Table 3.2: Generic Yosys synthesis cell summary 13
Table 3.3: Physical design summary (SKY130, sky130_fd_sc_hd) 13
Table 3.4: Setup and hold timing at all nine corners (all values in ns) 15

LIST OF ABBREVIATIONS

Abbreviation	Full form
AI	Artificial Intelligence
ASIC	Application-Specific Integrated Circuit
CTS	Clock Tree Synthesis
DRC	Design Rule Check
FF	Flip-Flop
FSM	Finite State Machine
GDS	Graphic Data System (GDSII layout format)
HAI	Healthcare-Associated Infection
LED	Light-Emitting Diode
LVS	Layout Versus Schematic
PDK	Process Design Kit
PVT	Process, Voltage and Temperature
RC	Resistance–Capacitance (interconnect parasitics)
RTL	Register Transfer Level
SDC	Synopsys Design Constraints
STA	Static Timing Analysis
TNS	Total Negative Slack
VLSI	Very Large Scale Integration
WHO	World Health Organization
WNS	Worst Negative Slack (worst slack)
tt / ss / ff	Typical / slow / fast process corner

LIST OF SYMBOLS

Symbol	Meaning	Unit
f_clk	System clock frequency	Hz
T_clk	Clock period	ns
DIV	Clocks per one-second tick	clocks
t_target	Selected wash duration (20, 30 or 40)	s
t_grace	Grace period for sensor dropout	ticks
N_good / N_bad	Good-wash and bad-wash counts (good_cnt, bad_cnt)	count
WNS	Worst slack at a timing corner	ns
TNS	Sum of all negative slacks	ns
V_DD	Supply voltage	V
µm	Micrometre (layout dimensions)	µm

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

Washing your hands properly is the cheapest and most effective way to slow the spread of infection in a hospital, and almost everybody agrees on that. What people do not always manage is to wash for long enough. A quick rinse under the tap feels like a wash, but soap and friction need time to do their work. The World Health Organization recommends 20 to 30 seconds for an alcohol-based hand rub and 40 to 60 seconds for washing with soap and water [1].

Healthcare-associated infections (HAIs) are infections that patients pick up while being treated for something else. Systematic reviews show that their burden is large, and larger in low-resource settings [2]. Studies of hand-hygiene compliance keep finding that real-world compliance falls well short of what the guidelines ask for [3]. The usual way hospitals find this out is by sending a trained observer to watch and count, but being watched changes behaviour, and one study found that compliance measured by observers was clearly higher than compliance measured electronically [4].

In this project I look at the problem from the electronics side. At a sink or a sanitiser dispenser, a sensor can tell us whether hands are present. A small digital circuit can use that single signal to time the wash, give live feedback with lights and a buzzer, decide at the end whether the wash was long enough, and keep a running count of good and bad washes. That circuit is what I designed.

I took the design all the way from an idea to a manufacturable layout: a written specification, synthesizable Verilog, a self-checking testbench, logic synthesis, place and route, static timing analysis at every corner, physical verification and a final GDSII file. Everything was done with open-source tools and the SkyWater 130 nm process design kit (PDK).

1.2 PROBLEM STATEMENT

People cut hand washing short for ordinary reasons: they are busy, nobody is watching, and the sink gives no feedback about how long they have been at it. Posters remind people but do not measure anything. Manual audits measure only a small sample of all washes and, as noted above, change the behaviour they are trying to observe.

Microcontroller-based or networked monitoring systems do work, but they bring software, power draw and cost that a plain sink dispenser may not justify.

The problem addressed: to design a compact, always-on, low-power digital circuit that takes a single hand-presence signal, times every wash, tolerates brief interruptions, reports compliance immediately, and records how many washes were good and how many were too short; and to carry that design through to a verified GDS layout.

1.3 OBJECTIVES

1. Define one specific function for the chip, with its ports, wash durations and grace period, before writing any RTL.
2. Write synthesizable Verilog for the synchronizer, debouncer, tick generator, wash controller and event counters.
3. Verify the design with a self-checking testbench that covers normal operation, reset behaviour, boundary conditions and corner cases.
4. Synthesize the design with Yosys and confirm there are no unintended latches.
5. Complete floorplanning, placement, clock-tree synthesis and routing with OpenLane on the SKY130 process and produce a GDS file.
6. Sign off timing (setup and hold) at all PVT and parasitic corners, together with DRC, LVS and antenna checks.
7. Document every design decision so that it can be explained and defended.

1.4 LITERATURE SURVEY

Hand-hygiene guidance and burden. The WHO guidelines on hand hygiene in health care set out when and how long staff should clean their hands, and give separate durations for alcohol-based rub and for soap and water [1]. Allegranzi et al. reviewed the evidence on endemic healthcare-associated infection and found that the burden is considerably higher in developing countries than in high-income ones [2]. These two sources give the project its reason to exist and its choice of wash durations.

Measuring compliance. Erasmus et al. reviewed studies of compliance with hand-hygiene guidelines in hospitals and reported average compliance of only about 40 per cent in the studies they covered [3]. Srigley et al. compared direct observation with an electronic monitoring system and showed a clear Hawthorne effect: people washed

more often when they knew an observer was present [4]. Together they argue for measurement that is automatic and does not depend on a person watching.

Digital design practice used in the chip. A signal that arrives from outside the clock domain, such as a hand sensor, must be passed through at least two flip-flops before it is used, otherwise a metastable value can spread into the logic [5]. Mechanical or noisy inputs also need debouncing, which in digital form means accepting a change only after it has been stable for a minimum time [6]. For timing, I follow the common practice of generating a one-cycle enable pulse instead of a slower derived clock, which keeps the whole design in one clock domain and makes static timing analysis simpler [5]. Timing concepts such as setup, hold and corner analysis are standard textbook material [11].

Open-source implementation flow. Yosys provides logic synthesis [7], OpenROAD provides floorplanning, placement, clock-tree synthesis, routing and timing analysis [8], and OpenLane 2 ties these together into one RTL-to-GDS flow [12] on top of the SkyWater SKY130 PDK [9]. This made it possible to complete the entire journey without any commercial licence.

In the material I reviewed, most work on hand-hygiene monitoring is about measuring and reporting compliance across a whole institution. This project sits at the very last step of that chain: the small circuit at the sink that actually does the timing and judging.

1.5 ORGANISATION OF THE REPORT

Chapter 2 explains the methodology: the specification, the tools, the architecture, each module, the verification plan and the implementation flow. Chapter 3 presents the results of simulation, synthesis, physical design and multi-corner timing analysis, and discusses cost, usefulness to society and environmental safety. Chapter 4 concludes the report and lists future work. The RTL listing, configuration files and commands needed to reproduce the results are in the appendices.

CHAPTER 2
METHODOLOGY

2.1 SYSTEM OVERVIEW

The chip is a controller. It does not contain the sensor, the lights or the buzzer; it reads one digital sensor signal and drives the outputs that those parts would use. In normal use the sequence of events is as follows.

1. The sensor input is first passed through a two-flip-flop synchronizer so that it is safe to use with the system clock.
2. A debounce counter ignores very short glitches, so only genuine hand presence is accepted.
3. When a stable “hands present” signal arrives, the controller starts a wash and latches the selected duration (20, 30 or 40 seconds). Changing the setting during the wash has no effect.
4. A tick generator produces one pulse per second, and a seconds counter counts up while hands are present. Three progress outputs switch on at about one third, two thirds and the full time.
5. If the sensor drops out, the controller moves to a paused state and starts a grace timer of about two seconds. If hands return in time, the wash carries on from where it stopped.
6. If the full time is reached, the controller raises the “done” output and the buzzer and adds one to the good-wash counter.
7. If the grace period runs out first, it raises the “too short” alarm and the buzzer and adds one to the bad-wash counter.
8. After the indication, the controller waits for the sensor to go low and returns to idle, ready for the next person.

2.2 SOFTWARE AND HARDWARE REQUIREMENTS

Because this is an ASIC design project, the “hardware” is the target silicon process, and almost all of the work is done with software tools. Table 2.1 lists the tools I used and what each one was for.

Table 2.1: Software tools used in the project

Tool	Version	Purpose
Verilog HDL	IEEE 1364-2005 [10]	Describing the synthesizable RTL and the testbench
Icarus Verilog	12.0	Compiling and simulating the design and testbench
Verilator	5.020	Linting the RTL for latches, width mismatches and style problems
Yosys	0.33	Generic logic synthesis and cell-count check
OpenLane	2.3.10	Complete RTL-to-GDS flow, run inside Docker
OpenROAD / OpenSTA	bundled with OpenLane	Floorplan, placement, CTS, routing and multi-corner STA
Magic, KLayout, Netgen	bundled with OpenLane	DRC, layout generation and LVS
KLayout (viewer)	0.30.12	Viewing the final GDS and taking layout screenshots
Docker	29.8.0	Running the OpenLane tool container
Python	3.12 (virtual environment)	Running OpenLane and rendering scripts
Git and GitHub Codespaces	cloud	Version control and the Linux development environment

Target technology. The design is implemented in the SkyWater SKY130 130 nm process using the high-density standard-cell library (sky130_fd_sc_hd) at a nominal supply of 1.8 V [9].

Off-chip components. The chip is meant to sit between a sensor and some simple indicators. I did not build this hardware; Table 2.2 lists what the chip would be connected to and why each part matters to the design.

Table 2.2: Off-chip components the chip is designed to work with

Component	Connected to	Why it is relevant
Hand-presence or water-flow sensor (digital output)	sensor_in	Tells the chip when washing is happening. It is asynchronous and can be noisy, which is why the synchronizer and debouncer exist.
Three LEDs (progress)	led_progress[2:0]	Show the person how far through the wash they are.
Green LED	led_done	Confirms a full-length wash.
Red LED	alarm_short	Warns that the wash ended too early.
Piezo buzzer	buzzer	Gives an audible result at the end of each wash.
Mode switch (2 bits)	mode_sel[1:0]	Selects 20, 30 or 40 second washes.

Component	Connected to	Why it is relevant
Push button	clear_log	Lets an authorised person clear the good and bad counters.
32.768 kHz watch crystal	clk	A very low-power clock that divides exactly to one second.
Small display or logger	good_cnt, bad_cnt	Shows or records the compliance counts.

2.3 DESIGN SPECIFICATION

I fixed the specification before writing any RTL. Table 2.3 gives the main design parameters and Table 2.4 gives the ports.

Table 2.3: Design parameters

Parameter	Value
System clock (intended)	32.768 kHz
One-second tick divider	32768 clocks (8 clocks in simulation)
Debounce time	2^11 = 2048 clocks, about 62.5 ms (4 clocks in simulation)
Wash durations (mode_sel)	00 = 20 s, 01 = 30 s, 10 = 40 s, 11 = 20 s (default)
Progress thresholds (20 s mode)	6 s, 13 s and 20 s
Grace period for sensor dropout	2 ticks (effective 1 to 2 s)
Result indication time	2 ticks
Log counters	two 8-bit saturating counters (hold at 255)
Reset	active-low, asynchronous

Table 2.4: Port description

Signal	Dir	Width	Description
clk	in	1	System clock
rst_n	in	1	Active-low asynchronous reset
sensor_in	in	1	Hands or water detected (asynchronous)
mode_sel	in	2	Selects the wash duration
clear_log	in	1	Clears the good and bad counters
led_progress	out	3	Wash progress in thirds
led_done	out	1	High when a wash was completed properly
alarm_short	out	1	High when a wash ended too early
buzzer	out	1	High during the result indication
good_cnt	out	8	Number of compliant washes
bad_cnt	out	8	Number of non-compliant washes

2.4 BLOCK DIAGRAM

Figure 2.1 shows how the blocks fit together. The sensor passes through the synchronizer and debouncer to the wash FSM. The tick generator gives the FSM its one-second timebase, and the FSM in turn holds the tick generator at zero while idle so that every wash starts with a full first second. The FSM sends a one-cycle pulse to one of the two saturating counters at the end of each wash.

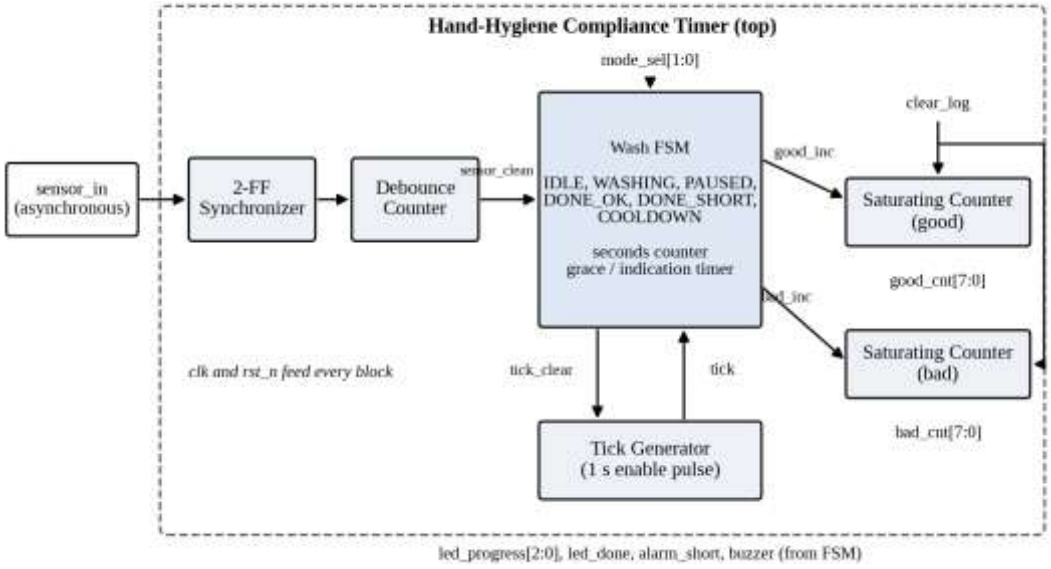


Figure 2.1: Block diagram of the hand-hygiene compliance timer

2.5 MODULE DESCRIPTION

2.5.1 Synchronizer

The sensor is not aligned to the system clock, so its edges can arrive at any moment, including exactly when a flip-flop is sampling. The first flip-flop may then settle to an unpredictable value (metastability). The second flip-flop samples the first one a full clock cycle later, by which time the value has almost certainly resolved. Both flip-flops reset to zero.

2.5.2 Debounce

The debouncer compares its input with its current output. While they are equal, its counter stays at zero. While they differ, the counter counts up, and only when it reaches its maximum does the output take the new value. If the input flips back before that, the counter is cleared, so short glitches never get through. At the intended 32.768 kHz clock a change has to be stable for about 62.5 ms. A pulse only a few clocks long is ignored completely.

2.5.3 Tick Generator

The tick generator counts clock cycles and raises a one-cycle-wide enable pulse once every DIV clocks. I deliberately made this an enable pulse rather than a divided clock. A derived clock would be a second clock domain that has to be constrained and balanced; an enable keeps one clock for the whole design. Its clear input is driven by the FSM and holds the counter at zero while the FSM is idle.

2.5.4 Wash FSM

The wash FSM is the heart of the design. It has six states (IDLE, WASHING, PAUSED, DONE_OK, DONE_SHORT, COOLDOWN), a six-bit seconds counter, and a second six-bit timer that is reused for the grace period and for the length of the result indication. In IDLE it latches mode_sel into an internal register, so that the target time cannot change in the middle of a wash. The target and the two progress thresholds are looked up from the latched mode. Outputs such as led_done, alarm_short and buzzer are decoded directly from the state, while good_inc and bad_inc are registered one-cycle pulses sent to the counters.

2.5.5 Saturating Counter

Each log counter is an eight-bit register that increments when its inc input is high and stops at 255 instead of wrapping to zero. The clear input has priority over inc. Two instances are used, one for good washes and one for bad washes, and both share the same clear_log input.

2.5.6 Top Level

The top module only connects the blocks. It exposes the tick divider and the debounce width as parameters. The hardware defaults are 32768 and 11, and the testbench overrides them with small values so that a complete wash simulates in a few hundred clock cycles instead of millions.

2.6 FSM AND FLOWCHART

Figure 2.2 shows the state diagram and Table 2.5 gives the behaviour of each state.

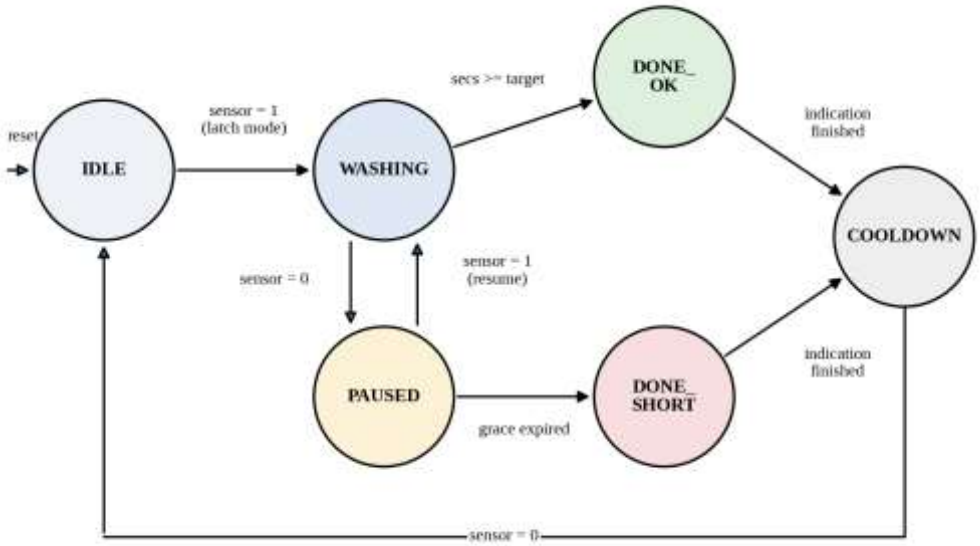


Figure 2.2: State diagram of the wash FSM

Table 2.5: FSM state table

State	Condition	Next state	Action / outputs
IDLE	sensor = 1	WASHING	Latch mode_sel; timers are held at zero while idle
WASHING	sensor = 0	PAUSED	Start grace timer
WASHING	secs >= target	DONE_OK	Pulse good_inc
WASHING	tick	WASHING	secs = secs + 1; progress outputs update
PAUSED	sensor = 1	WASHING	Resume; seconds already counted are kept
PAUSED	grace ticks reached	DONE_SHORT	Pulse bad_inc
DONE_OK	indication finished	COOLDOWN	led_done and buzzer on
DONE_SHORT	indication finished	COOLDOWN	alarm_short and buzzer on
COOLDOWN	sensor = 0	IDLE	All indications off

2.7 DESIGN DECISIONS

A few decisions shaped the design, and each one has a reason behind it.

- **One clock domain.** Everything runs on one clock. The only asynchronous input that matters is the sensor, and it goes through the synchronizer.
- **Tick enable instead of a divided clock.** This avoids gated or derived clocks, which makes clock-tree synthesis and timing analysis simpler.
- **Mode latched at the start of a wash.** A person leaning on the mode switch cannot extend or shorten a wash that is already running.

- **Grace period.** Hands naturally leave the sensor for a moment while soap is applied or the tap is adjusted. Without a grace period every such moment would count as a failed wash.
- **Saturating counters.** A log that wraps from 255 back to 0 would hide a lot of bad washes. Holding at 255 is the safer failure.
- **Parameterised timing.** The one-second divider and the debounce width are parameters so that simulation stays fast while the hardware keeps its real values.
- **Asynchronous reset.** The reset is asserted asynchronously and treated as a false path in timing analysis. Synchronising its release is listed as future work.

2.8 VERIFICATION METHODOLOGY

I wrote a self-checking testbench that drives sensor_in like a person at a sink and compares the outputs with expected values automatically, printing PASS or FAIL for each check. One simulated "second" is 8 clock cycles and the debounce time is 4 clock cycles. The expected values come from the specification, not from watching what the RTL does, otherwise a bug in the RTL would simply be copied into the test.

The testbench covers a full compliant wash in each mode, a too-short wash, dropouts shorter and longer than the grace period, sensor glitches, a mode change in the middle of a wash, the progress outputs, reset in the middle of a wash, the clear_log input and two washes back to back. Two continuous assertions run throughout: led_done and alarm_short are never high together, and the counters never decrease except on reset or clear_log. A watchdog stops the simulation if it ever hangs.

2.9 RTL-TO-GDS IMPLEMENTATION FLOW

Figure 2.3 summarises the flow. After the RTL passed lint and simulation, I ran Yosys to check the cell count and look for latches. I then ran the full OpenLane flow, which performs synthesis, floorplanning, placement, clock-tree synthesis, routing, parasitic extraction, multi-corner timing analysis, DRC, LVS and antenna checks, and finally writes the GDS.

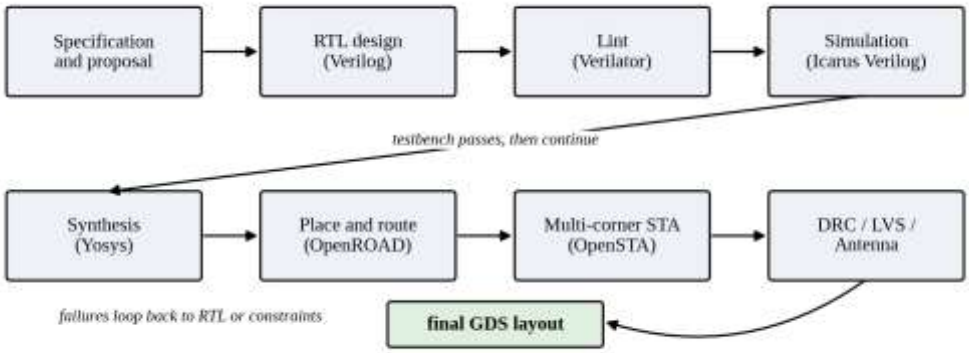


Figure 2.3: RTL-to-GDS design flow

The first OpenLane run used a relaxed 50 ns clock, only to confirm that the design could reach GDS. I then wrote my own constraints file and tightened the clock to 20 ns (50 MHz). Table 2.6 lists the main configuration settings and the constraints used.

Table 2.6: OpenLane configuration and timing constraints

Item	Setting
Design name / clock port	top / clk
Clock period	20 ns (50 MHz)
Clock uncertainty / transition	0.25 ns / 0.15 ns
Input delay (sensor_in, mode_sel, clear_log)	10 ns relative to clk
Output delay (all outputs)	10 ns relative to clk
rst_n	false path (asynchronous reset)
Core utilisation / placement density	40 % / 50 %
Timing corners analysed	3 PVT (tt 25 °C 1.80 V, ss 100 °C 1.60 V, ff −40 °C 1.95 V) × 3 RC (nom, min, max) = 9

2.10 USE OF AI ASSISTANCE

I used an AI assistant as a helper, not as a replacement for understanding. The proposal, ports and state table were finalised first, and RTL was requested one small module at a time. Each module was read line by line and linted with Verilator, which caught real issues: missing end-of-file newlines and width mismatches in the tick generator (WIDTHEXPAND and WIDTHTRUNC), which I fixed. The testbench expectations were checked against my own specification. Simulation, synthesis and physical-design results in this report come from tools I ran myself.

CHAPTER 3
RESULTS AND DISCUSSION

3.1 FUNCTIONAL SIMULATION RESULTS

All 16 checks in the self-checking testbench passed with 0 failures, and neither continuous assertion fired. Table 3.1 lists the checks.

Table 3.1: Testbench checks and results

ID	Test	Stimulus	Expected	Result
T1	Reset state	Reset applied	Counters 0, no indications	PASS
T2	Full wash, 20 s	Sensor high for 23 s	good_cnt + 1, bad_cnt unchanged	PASS
T3	Too-short wash	8 s, then leave for 5 s	bad_cnt + 1, good_cnt unchanged	PASS
T4	Brief dropout	Drop for 6 clocks (below grace), return	Wash completes, good_cnt + 1	PASS
T5	Sensor glitch	2-clock pulse	Ignored, no progress	PASS
T6a	30 s mode, early	25 s of washing	Not yet done	PASS
T6b	30 s mode, complete	After 30 s	good_cnt + 1	PASS
T7a	40 s mode, early	35 s of washing	Not yet done	PASS
T7b	40 s mode, complete	After 40 s	good_cnt + 1	PASS
T8	Mode change mid-wash	Switch to 40 s after 5 s	Wash still ends at 20 s	PASS
T9a	Progress, one third	About 8 s	led_progress = 001	PASS
T9b	Progress, two thirds	About 14 s	led_progress = 011	PASS
T10	Reset mid-wash	Reset after 6 s	Counters 0, progress 000	PASS
T11a	Before clear	After a good wash	good_cnt nonzero	PASS
T11b	clear_log	Pulse clear_log	Both counters 0	PASS
T12	Back-to-back washes	Two full washes	good_cnt + 2 exactly	PASS

Figure 3.1 shows the simulated waveform of a complete wash.

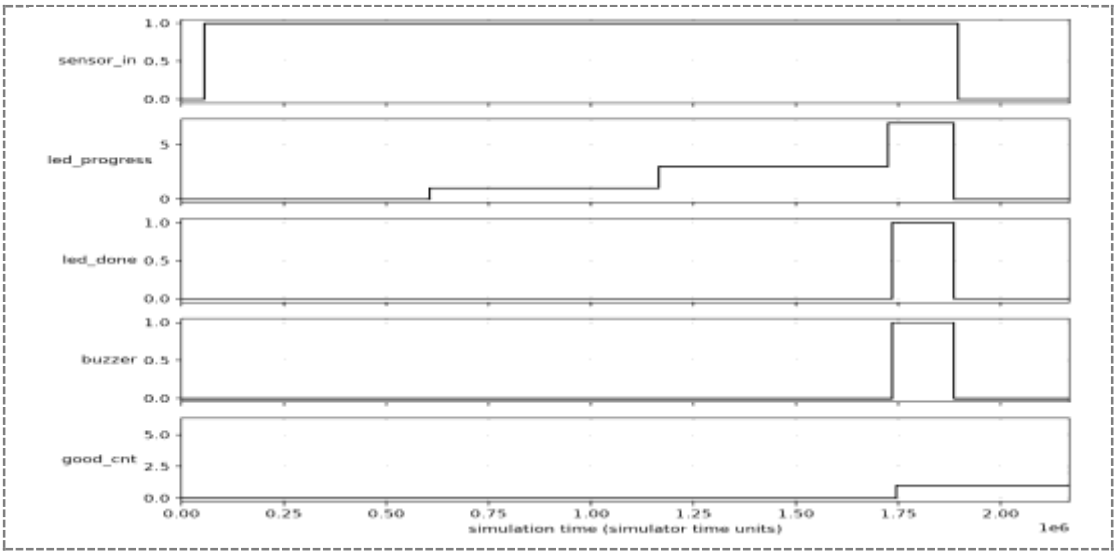


Figure 3.1: Simulated waveform of a compliant wash

3.2 SYNTHESIS RESULTS

Yosys synthesised the complete design into 457 generic cells. No latches were inferred, no memories were used, and the design contains 65 flip-flops (34 with an enable and 31 without). Table 3.2 gives the breakdown.

Table 3.2: Generic Yosys synthesis cell summary

Cell type	Count	Cell type	Count
$_ANDNOT_	155	$_NOR_	13
$_AND_	16	$_NOT_	15
$_DFFE_PN0P_ (flip-flop with enable)	34	$_ORNOT_	19
$_DFF_PN0_ (flip-flop)	31	$_OR_	70
$_MUX_	1	$_XNOR_	28
$_NAND_	41	$_XOR_	34
Total cells	**457**	**Total flip-flops**	**65**

The flip-flop count matches what I expected from the architecture: 2 in the synchronizer, 11 in the debouncer plus its output, about 20 in the tick generator and about 19 in the FSM, and 16 in the two log counters.

3.3 PHYSICAL DESIGN RESULTS

The OpenLane flow completed all 78 stages in about two minutes on a two-core cloud machine. Table 3.3 summarises the result.

Table 3.3: Physical design summary (SKY130, sky130_fd_sc_hd)

Item	Result
Flow	OpenLane 2.3.10, 78 / 78 stages complete
Clock period	20 ns (50 MHz)

Item	Result
Devices and nets in layout (Netgen)	364 devices, 364 nets (includes tap, fill and decap cells)
Flip-flops (sky130_fd_sc_hd__dfrtp)	65 (52 × _1, 9 × _2, 4 × _4)
Hold-fix delay cells (dlygate4sd3_1)	39, inserted automatically
Die size	about 104.29 µm × 115.01 µm (about 0.012 mm²)
Setup / hold violations	None
Max slew / max capacitance violations	None
DRC / LVS / Antenna	Passed / Passed / Passed
LVS result	Circuits match uniquely

The 39 delay cells are worth a comment. Many paths in this design are short (a flip-flop feeding a flip-flop through a little logic), and at the fast corner they arrive so early that they threaten the hold requirement. The tool therefore added delay buffers on its own to fix this.

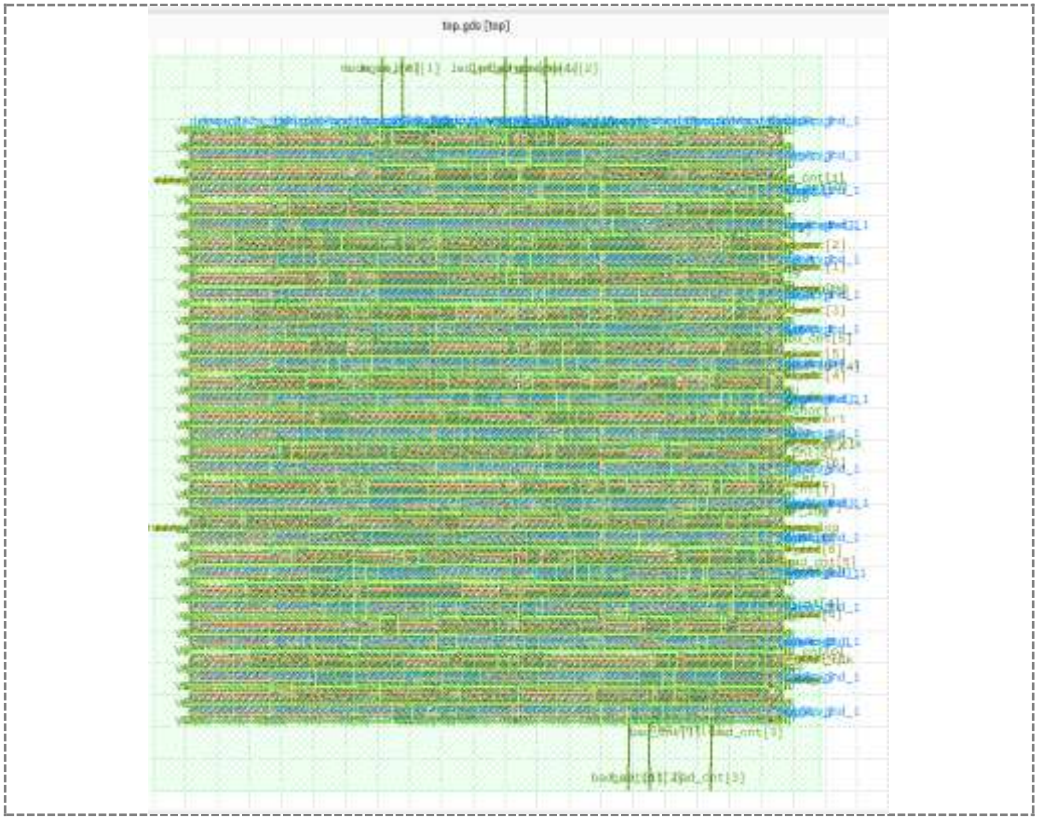


Figure 3.2: Final GDS layout of the design (full die)

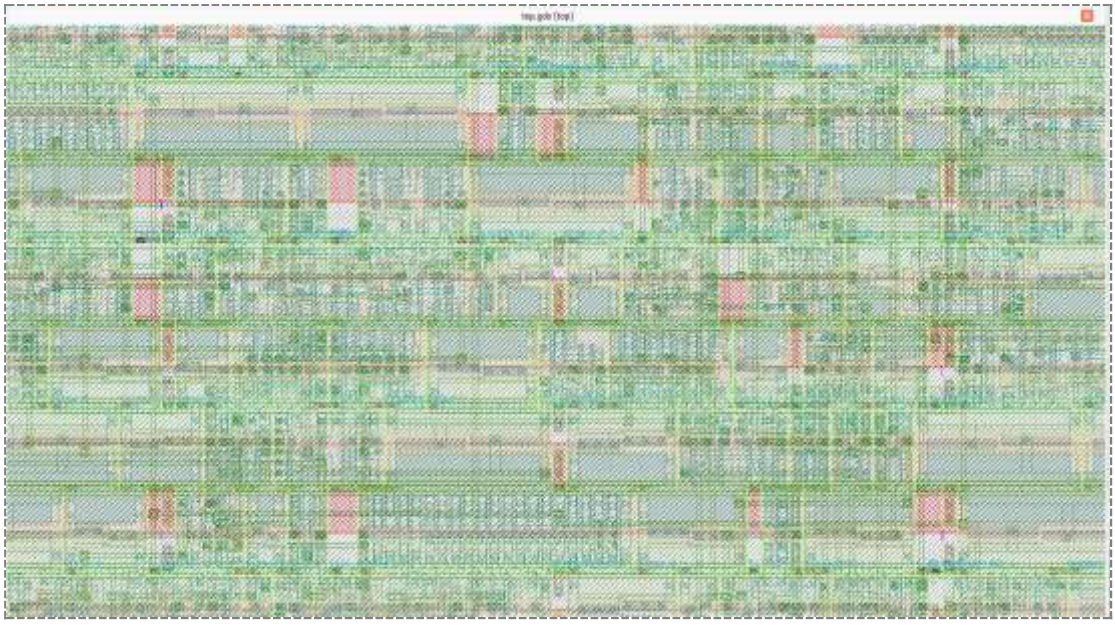


Figure 3.3: Zoomed view of the layout showing cell rows and routing

3.4 MULTI-CORNER STATIC TIMING ANALYSIS

Timing was checked at nine corners: three process/voltage/temperature (PVT) conditions, each with nominal, minimum and maximum interconnect parasitics. Slow-corner (ss) analysis is the one that matters for setup, because gates are slowest there. Fast-corner (ff) analysis matters for hold, because data moves fastest and may arrive before the capturing flip-flop is ready. Table 3.4 gives the worst setup and hold slack and the total negative slack (TNS) at every corner.

Table 3.4: Setup and hold timing at all nine corners (all values in ns)

Corner	Setup WNS	Setup TNS	Hold WNS	Hold TNS
nom_tt_025C_1v80	8.018	0	0.314	0
nom_ss_100C_1v60	6.389	0	0.885	0
nom_ff_n40C_1v95	8.645	0	0.106	0
min_tt_025C_1v80	8.026	0	0.312	0
min_ss_100C_1v60	6.414	0	0.883	0
min_ff_n40C_1v95	8.660	0	0.105	0
max_tt_025C_1v80	8.006	0	0.316	0
max_ss_100C_1v60	6.346	0	0.887	0
max_ff_n40C_1v95	8.626	0	0.107	0
Overall worst	**6.346** (max_ss)	**0**	**0.105** (min_ff)	**0**

Every corner has positive setup and hold slack and a TNS of zero. As expected, setup slack is smallest at the slow corner (6.346 ns, max_ss) and hold slack is smallest at the

fast corner (0.105 ns, min_ff). The hold margin is small but positive, which is exactly why the flow inserted the delay cells.

3.5 SIGN-OFF CHECKS

- **DRC:** the final layout has no design-rule violations.
- **LVS:** the layout and the synthesized netlist match uniquely, with the same number of devices and nets in both.
- **Antenna:** no antenna violations.
- **Electrical:** no maximum-slew or maximum-capacitance violations.

3.6 DISCUSSION

Real clock versus analysis clock. The chip is intended to run from a 32.768 kHz crystal, which gives an exact one-second tick, a clock period of about 30.5 µs. Timing was signed off at 50 MHz, about 1500 times faster. Passing at 50 MHz with more than 6 ns of setup slack in the slowest corner means the circuit has a very large margin at its real operating frequency. I chose the faster clock so that the timing report would be a meaningful test and not a trivial pass.

Wash duration against the WHO figures. The 20 s and 30 s modes match the WHO range for alcohol-based hand rub, and the 40 s mode matches the lower end of the WHO range for soap and water [1]. A 60 s mode would be a sensible addition for full soap-and-water washing.

Known limitations. I found several things worth being honest about.

- The grace period is accurate only to within about a second. The tick counter keeps running while the sensor is low, so the first grace tick can arrive early. A two-tick grace therefore lasts between one and two seconds.
- If the sensor drops in the very same cycle that the seconds counter reaches its target, the sensor check has priority and the FSM moves to PAUSED. The wash is then only completed if hands return within the grace period.
- The counters are ordinary registers and lose their values when power is removed. A real product would need non-volatile storage or an external logger.
- The reset is asynchronous on release as well as on assertion. A reset synchroniser would make it more robust.

- Power was not analysed in detail in this project.

3.7 COST ANALYSIS

Every tool I used is free and open source, so the design cost no licence fees. The silicon itself is extremely small (about 0.012 mm^2), so at volume the cost of the chip would be a very small part of a dispenser. The bill of materials would be dominated by the sensor, the buzzer and LEDs, the housing and the battery. I have not obtained a fabrication quote; for prototypes, low-cost multi-project shuttles on open PDKs would be the natural route.

3.8 USEFULNESS TO SOCIETY

Healthcare-associated infections cause avoidable suffering and cost [2], and poor hand-hygiene compliance is a well-documented contributor [3]. A cheap device at the sink that gives instant feedback and keeps an honest count could help in hospitals and clinics, and also in school toilets, kitchens and food-processing plants. Because it measures automatically, it avoids the observer bias seen in manual audits [4]. Because it needs no software or operating system, it can be very cheap, reliable and long-lasting.

3.9 ENVIRONMENTAL SAFETY

The circuit is tiny and runs at a low supply voltage (1.8 V for the core), so it can be powered for a long time from a small battery, which reduces battery waste. It has no radio, no moving parts and no mains connection, so it adds no electromagnetic or electrical-shock concerns of its own. A small die uses very little silicon. Encouraging complete hand washing does not require any extra chemicals beyond what is already used.

CHAPTER 4
CONCLUSION AND FUTURE SCOPE

4.1 CONCLUSION

I designed a hand-hygiene compliance timer as a small digital circuit and took it from a written specification to a signed-off GDS layout. The design synchronises and debounces a sensor input, times a wash in three selectable durations, tolerates short interruptions, signals the result with progress, done and alarm outputs, and keeps saturating counts of good and bad washes.

The testbench passed all 16 checks. Yosys found no latches and 65 flip-flops. The OpenLane flow produced a layout of about 104 µm × 115 µm that passes DRC, LVS and antenna checks, and static timing analysis at all nine corners shows positive setup and hold slack with zero total negative slack: the worst setup slack is 6.346 ns and the worst hold slack is 0.105 ns at a 20 ns clock.

The project also taught me that passing simulation is only the start. The linter, the hold-fix cells and the corner-by-corner timing report each showed me something that simulation alone would never have revealed.

4.2 FUTURE SCOPE

- Add the full six-to-eight step WHO hand-wash technique as a step indicator that changes every few seconds.
- Add a 60 s mode and make the durations configurable through a small register interface (SPI or I²C).
- Add non-volatile or externally readable logging (a UART or SPI output), and timestamps from a real-time clock.
- Add staff identification so washes can be linked to individuals, with appropriate privacy safeguards.
- Add a reset synchroniser, and run gate-level simulation with timing back-annotation and a detailed power analysis.
- Prototype the design on an FPGA with a real sensor, and submit it to a multi-project shuttle for fabrication.

REFERENCES

[1] World Health Organization – 'WHO Guidelines on Hand Hygiene in Health Care: First Global Patient Safety Challenge, Clean Care Is Safer Care', Publisher: WHO Press, Place: Geneva, Country: Switzerland, Year: 2009.

[2] B. Allegranzi, S. Bagheri Nejad, C. Combescure, W. Graafmans, H. Attar, L. Donaldson, D. Pittet – 'Burden of endemic health-care-associated infection in developing countries: systematic review and meta-analysis', The Lancet, Year: 2011, Vol. 377, Iss. 9761, pp. 228 to 241.

[3] V. Erasmus, T. J. Daha, H. Brug, J. H. Richardus, M. D. Behrendt, M. W. Vos, E. F. van Beeck – 'Systematic review of studies on compliance with hand hygiene guidelines in hospital care', Infection Control & Hospital Epidemiology, Year: 2010, Vol. 31, Iss. 3, pp. 283 to 294.

[4] J. A. Srigley, C. D. Furness, G. R. Baker, M. Gardam – 'Quantification of the Hawthorne effect in hand hygiene compliance monitoring using an electronic monitoring system: a retrospective cohort study', BMJ Quality & Safety, Year: 2014, Vol. 23, Iss. 12, pp. 974 to 980.

[5] C. E. Cummings – 'Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog', Synopsys Users Group (SNUG) Conference, Boston, USA, Year: 2008.

[6] J. Ganssle – 'A Guide to Debouncing', The Ganssle Group, Year: 2004 (revised 2008).

[7] C. Wolf, J. Glaser – 'Yosys – A Free Verilog Synthesis Suite', Proceedings of the 21st Austrian Workshop on Microelectronics (Austrochip), Year: 2013.

[8] T. Ajayi, D. Blaauw, T.-B. Chan, C.-K. Cheng, et al. – 'Toward an Open-Source Digital Flow: First Learnings from the OpenROAD Project', Proceedings of the 56th ACM/IEEE Design Automation Conference (DAC), Year: 2019.

[9] SkyWater Technology and Google – 'SkyWater Open Source PDK (SKY130) Documentation', https://skywater-pdk.readthedocs.io.

[10] IEEE – 'IEEE Standard for Verilog Hardware Description Language', IEEE Std 1364-2005, Publisher: IEEE, Year: 2006.

[11] N. H. E. Weste, D. M. Harris – 'CMOS VLSI Design: A Circuits and Systems Perspective', 4th Edition, Publisher: Addison-Wesley, Country: USA, Year: 2011.

[12] OpenLane Project – 'OpenLane 2 Documentation', https://openlane2.readthedocs.io.

APPENDIX A

wash_fsm.v (main control module)

```
// Wash FSM: times a hand wash, tolerates short dropouts, reports result.
module wash_fsm #(
    parameter [5:0] GRACE_S = 6'd2,   // allowed sensor dropout (seconds)
    parameter [5:0] BEEP_S  = 6'd2    // result indication time (seconds)
) (
    input  wire       clk,
    input  wire       rst_n,
    input  wire       sensor,         // debounced hands-present signal
    input  wire       tick,           // 1-cycle pulse once per second
    input  wire [1:0] mode_sel,
    output wire       tick_clear,
    output reg  [2:0] led_progress,
    output wire       led_done,
    output wire       alarm_short,
    output wire       buzzer,
    output reg        good_inc,
    output reg        bad_inc
);
    localparam [2:0] IDLE       = 3'd0,
                     WASHING    = 3'd1,
                     PAUSED     = 3'd2,
                     DONE_OK    = 3'd3,
                     DONE_SHORT = 3'd4,
                     COOLDOWN   = 3'd5;

    reg [2:0] state;
    reg [1:0] mode_r;               // mode latched at wash start
    reg [5:0] secs;                 // seconds washed
    reg [5:0] aux;                  // grace / indication timer

    // Target and progress thresholds for the latched mode
    reg [5:0] target, t1, t2;
    always @* begin
        case (mode_r)
            2'b01:   begin target = 6'd30; t1 = 6'd10; t2 = 6'd20; end
            2'b10:   begin target = 6'd40; t1 = 6'd13; t2 = 6'd26; end
            default: begin target = 6'd20; t1 = 6'd6;  t2 = 6'd13; end
        endcase
    end

    // State register and timers
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state    <= IDLE;
            mode_r   <= 2'b00;
            secs     <= 6'd0;
            aux      <= 6'd0;
            good_inc <= 1'b0;
            bad_inc  <= 1'b0;
        end else begin
            good_inc <= 1'b0;
            bad_inc  <= 1'b0;
            case (state)
                IDLE: begin
                    secs <= 6'd0;
                    aux  <= 6'd0;
                    if (sensor) begin
                        mode_r <= mode_sel;
                        state  <= WASHING;
                    end
                end

                WASHING: begin
                    if (!sensor) begin
                        aux   <= 6'd0;
                        state <= PAUSED;
```

```
            end else if (secs >= target) begin
               aux      <= 6'd0;
               good_inc <= 1'b1;
               state    <= DONE_OK;
            end else if (tick) begin
               secs <= secs + 6'd1;
            end
         end

         PAUSED: begin
            if (sensor) begin
               state <= WASHING;
            end else if (tick) begin
               if ((aux + 6'd1) >= GRACE_S) begin
                  aux     <= 6'd0;
                  bad_inc <= 1'b1;
                  state   <= DONE_SHORT;
               end else begin
                  aux <= aux + 6'd1;
               end
            end
         end

         DONE_OK, DONE_SHORT: begin
            if (tick) begin
               if ((aux + 6'd1) >= BEEP_S) state <= COOLDOWN;
               else                        aux   <= aux + 6'd1;
            end
         end

         COOLDOWN: begin
            if (!sensor) state <= IDLE;
         end

         default: state <= IDLE;
      endcase
   end
end

// Outputs
assign tick_clear  = (state == IDLE);
assign led_done    = (state == DONE_OK);
assign alarm_short = (state == DONE_SHORT);
assign buzzer      = (state == DONE_OK) || (state == DONE_SHORT);

always @* begin
   if (state == IDLE || state == COOLDOWN)
      led_progress = 3'b000;
   else
      led_progress = {(secs >= target), (secs >= t2), (secs >= t1)};
end
endmodule
```

APPENDIX B

OpenLane configuration (config/config.json)

```
{
 "DESIGN_NAME": "top",
 "VERILOG_FILES": [
   "dir::../src/synchronizer.v",
   "dir::../src/debounce.v",
   "dir::../src/tick_gen.v",
   "dir::../src/sat_counter.v",
   "dir::../src/wash_fsm.v",
   "dir::../src/top.v"
 ],
 "CLOCK_PORT": "clk",
 "CLOCK_PERIOD": 20,
 "PNR_SDC_FILE": "dir::top.sdc",
 "SIGNOFF_SDC_FILE": "dir::top.sdc",
 "FP_SIZING": "relative",
 "FP_CORE_UTIL": 40,
 "PL_TARGET_DENSITY_PCT": 50,
 "RUN_LINTER": true,
 "DESIGN_IS_CORE": false
}
```

Timing constraints (config/top.sdc)

```
set clk_port clk
create_clock -name clk -period 20 [get_ports $clk_port]
set_clock_uncertainty 0.25 [get_clocks clk]
set_clock_transition 0.15 [get_clocks clk]

set_input_delay  10 -clock clk [get_ports {sensor_in mode_sel[*] clear_log}]
set_output_delay 10 -clock clk [get_ports {led_progress[*] led_done buzzer alarm_short good_cnt[*] bad_cnt[*]}]
set_false_path -from [get_ports rst_n]
```

APPENDIX C

Commands to reproduce the results

```
# Lint
verilator --lint-only -Wall --top-module top src/*.v

# Simulate
iverilog -g2012 -o test/tb_top.out test/tb_top.v src/*.v
vvp test/tb_top.out

# Generic synthesis
yosys -p "read_verilog -sv src/*.v; hierarchy -check -top top; synth -top top -flatten; check; stat"

# RTL-to-GDS (inside the Python virtual environment)
python -m openlane --dockerized config/config.json

# Per-corner timing metrics
grep -E "timing__(setup|hold)__(ws|tns)" docs/results/metrics_final.csv
```